Introduction

In this programming assignment, you will delve into the powerful world of regular expressions. Regular expressions, often abbreviated as regex, enable you to define flexible patterns for matching and manipulating text. We will use Python's regular expression package (`re`), which provides a language for expressing patterns that match sets of strings.

This assignment aims to reinforce your understanding of regexp, substitution, and capture groups, providing hands-on experience in implementing a practical text processing utility. The tool will not only showcase your mastery of regex concepts but also allow users to interactively define and apply their custom transformation rules. Get ready to explore the versatility of regular expressions and create a tool that can intelligently transform text based on user-defined patterns.

A set of files you will need for this assignment are available in the folder `data`.

In [18]:
```
import re
```

Task 1: Warm-up

The following are warm-up exercises to get some practice with regular expressions. For these exercises you'll be searching from a file called `names.txt` that contains both single-word usernames as well as full names. Let's first load and store data from this file:

In [19]:
```
with open(r'C:\Users\hasan\Downloads\3.1P-resources\3.1P - Regular Expressions\data\names.txt', 'r') as file:
        names = file.read()
print(names)
```

```
terribleSnail
abc123
condemnedInstructor
messsage
keep_it_simple_stupid
Zoran L Fang
myUsername210
abc
asdf
joshy
Hank Lazowska
123feralHog
Red Panda5
1
abc1
DIRT
*star*
oshie poshy
0shie p0shy
OSHIE POSHY
O Posh
Oshie poshy
oshie Poshy
```

Problem 1

Write a regex that finds all usernames that contain at least one numeric character.

In [20]:
```
def find_usernames_with_numeric(names):
    ## START YOU CODE HERE
    pattern = re.compile(r".*\d.*")
    ## END
    return [name for name in names.splitlines() if pattern.match(name)]

assert find_usernames_with_numeric(names) == ['abc123', 'myUsername210', '123feralHog', 'Red Panda5', '1', 'abc1', '0shie
print("Success!")
```

```
Success!
```

Problem 2

Write a regex to find all usernames that are exactly 4 characters long and consist only of alphabetic characters.

In [21]:
```
def find_4_char_usernames(names):
    ## START YOU CODE HERE
    pattern = re.compile(r"^[A-Za-z]{4}$")
    ## END
    return [name for name in names.splitlines() if pattern.match(name)]

assert find_4_char_usernames(names) == ['asdf', 'DIRT'], "Test failed!"
print("Success!")
```

```
Success!
```

Problem 3

Write a regex to find all first and last names. A name is an uppercase letter followed by one or more lowercase letters and first and last names are separated by a single space. Your command should not match against names that contain a middle name or initials.

In [22]:
```
def find_first_last_names(names):
    ## START YOU CODE HERE
    pattern = re.compile(r"^[A-Z][a-z]+ [A-Z][a-z]+$")
    ## END
    return [name for name in names.splitlines() if pattern.match(name)]

assert find_first_last_names(names) == ['Hank Lazowska'], "Test failed!"
print("Success!")
```

```
Success!
```

Task 2: More RegEx

After a few weeks in your new role, the management has identified a critical need for our business to actively engage in product sales. Your key responsibility will be to contribute to the development of regular expressions that facilitate customer account creation and billing processes. Your task involves using regex patterns for each item below, ensuring they effectively perform the specified operations. Your expertise in crafting these regular expressions will be vital in optimizing customer management and billing systems.

Problem 4: Email Validation

As part of account creation, customers must provide a valid email address. Write a regex that prints out all valid email addresses. For the purposes of this assignment, we will use as simplified version of valid email address is defined as follows:

- A username between 1 and 16 lowercase letters, uppercase letters, or digits
- An `@` symbol
- A domain (e.g., `gmail`) that consists of any number of lowercase letters
- A period
- A tld (top level domain, e.g., `.com`) that consists of 2 or more lowercase letters

For these task you'll be using a file called `emails.txt` that contains both email addresses. Let's first load and store emails from this file:

In [23]:
```
with open(r'C:\Users\hasan\Downloads\3.1P-resources\3.1P - Regular Expressions\data\emails.txt', 'r') as file:
        emails = file.read()
print(emails)
```

```
jle@uw.edu
k@gmailcom
hissingDonkey567@yahoo.com
342highground@council.com
aszotatgmail.co
hopkins@apple.COM
justchilling@cedarrapids|com
perkins8@uw.com
colorfulcupcake
fungfangfaang@agar.io
joshmyrealemail@imake.tools
asuperduperuberlongemail@miles.long
```

In [24]:
```
def find_valid_emails(emails):
    ## START YOU CODE HERE
    pattern = re.compile(r"^[A-Za-z0-9]{1,16}@[a-z]+\.[a-z]{2,}$")
    ## END
    return [email for email in emails.splitlines() if pattern.match(email)]

assert find_valid_emails(emails) == ['jle@uw.edu','hissingDonkey567@yahoo.com','342highground@council.com',
                                     'perkins8@uw.com','fungfangfaang@agar.io','joshmyrealemail@imake.tools'], "Test faile
print("Success!")
```

```
Success!
```

Problem 5: Password Validation

Customers also must choose a strong password when creating an account. Write a regex that prints out all valid passwords. A valid password is defined as follows:

- Must be at least 8 characters long
- Must contain at least one uppercase character
- Must contain at least one lowercase character
- Must contain at least one digit.

For this task you'll be using a file called `passwords.txt` that contains both passwords. Let's first load and store passwords from this file:

In [25]:
```
with open(r'C:\Users\hasan\Downloads\3.1P-resources\3.1P - Regular Expressions\data\passwords.txt', 'r') as file:
        passwords = file.read()
print(passwords)
```

```
missingAnyNumbers
0123456
01234567
alllowercase
ALLUPPERCASE
Apples42
orangE17
1cupCake
asdf
720
glhf56
210afk
A9i
```

In [26]:
```
def find_valid_passwords(passwords):
    ## START YOU CODE HERE
    pattern = re.compile(r"^(?=.*[A-Z])(?=.*[a-z])(?=.*\d).{8,}$")
    ## END
    return [password for password in passwords.splitlines() if pattern.match(password)]

assert find_valid_passwords(passwords) == ['Apples42', 'orangE17', '1cupCake'], "Test failed!"
print("Success!")
```

```
Success!
```

Problem 6: Credit Card validation

In order to process payments for orders, you need to validate credit cards entered on the website. Since we are a small company, we will only be accepting Visa and Mastercard. Write a regex that identifies all well-formatted credit cards that we accept. The format of these cards are as follows:

- Master Card
  - Begins with a 5
  - Exactly 16 digits long, grouped into sets of 4 separated by a space
- Visa
  - Begins with a 4
  - Between 13 and 16 digits long, grouped into sets of 4 (last may be shorter) separated by a space

For this task you'll be using a file called `cards.txt` that contains credit cards numbers. Let's first load and store cards from this file:

In [27]:
```
with open(r'C:\Users\hasan\Downloads\3.1P-resources\3.1P - Regular Expressions\data\cards.txt', 'r') as file:
        cards = file.read()
print(cards)
```

```
5123 4567 8910 1112
5123456789101112
3908 0978 0890 7890
4789 2784 8910 8290
4989 9321 1230 9
5419 2933 1839 17
4740 1839 1281
4789 0870 8908 70987
58907890782309171
5192 9295 91828818
4182 2884 1232 9582 2182
```

In [28]:
```
def find_valid_cards(cards):
    ## START YOU CODE HERE
    pattern = re.compile(r"^(5\d{3}( \d{4}){3}|4\d{3}( \d{4}){3}|4\d{3}( \d{4}){2} \d{1,4})$")
    ## END
    return [card for card in cards.splitlines() if pattern.match(card)]

assert find_valid_cards(cards) == ['5123 4567 8910 1112', '4789 2784 8910 8290', '4989 9321 1230 9'], "Test failed!"
print("Success!")
```

```
Success!
```

Congratulations!

Congratulations on completing the assignment! Your dedication and effort are commendable. By successfully working through the coding exercises and written exercises, you have demonstrated a strong understanding of the concepts and principles related to regular expressions.

Congratulations on finishing this notebook!

Acknowledgement

About the Author

This notebook was authored by Mohamed Reda Bouadjenek. He is a Senior Lecturer (Assistant Professor) of Applied Artificial Intelligence in the School of Information Technology at Deakin University, Australia.

Contact Information

- **Email:** reda.bouadjenek@deakin.edu.au
- **GitHub:** https://github.com/rbouadjenek/